

Individual Project Report

Multi-Label Face Attribute Classification using DenseNet169 + BiLSTM

George Washington University | CSCI 6260 Deep Learning | Spring 2026

Yuan (Irene) Xia | G50123456

1. Introduction and Shared Work Overview

This project builds a deep learning pipeline for conditional face attribute generation and evaluation using the CelebA dataset, which contains 202,599 celebrity images annotated with 40 binary facial attributes. The overall system consists of three components developed across the group: a multi-label attribute classifier (my contribution), a Conditional Variational Autoencoder (CVAE) for image generation (Zach's contribution), and an interactive Streamlit web demonstration (teammate's contribution).

Component	Developer	Description
Multi-label Classifier (framework.py)	Irene Xia (me)	DenseNet169 + BiLSTM classifier for all 40 attributes; complete training pipeline including preprocessing, disk caching, SWA, threshold tuning, and output generation
CVAE (cvae.py + cvae_train.py)	Zachary Cardell	Conditional VAE architecture, training loop, and image generation
Streamlit Demo	Jiatong Peng	Web interface integrating both models for interactive generation and evaluation
Report Writing	All members	Each wrote individual report; group report compiled jointly

Table 1. Division of work across the group.

The pipeline workflow is: (1) Train the classifier on real CelebA images; (2) Train the CVAE to generate images conditioned on attribute vectors; (3) Pass generated images through the classifier to evaluate generation quality; (4) Display results in the Streamlit demo. My individual contribution covers step (1) entirely — the complete classifier system in framework.py.

2. Description of Individual Work and Background

My contribution is the design and implementation of the multi-label face attribute classifier (framework.py). This section provides technical background on the algorithms involved.

2.1 Problem Formulation: Multi-Label Classification

Given a face image x , the goal is to predict a binary vector $y \in \{0,1\}^{40}$ where $y_c = 1$ if attribute c is present. This is a multi-label problem: multiple attributes can be simultaneously present. It differs from multi-class classification (exactly one class applies) and requires an independent binary decision per attribute.

The main challenges are: (1) severe class imbalance — positive rates range from 83.2% (No_Beard) to 2.1% (Bald); (2) attribute correlation — Male and No_Beard frequently co-occur; (3) label subjectivity — Attractive and Oval_Face depend on annotator judgment.

2.2 DenseNet169

DenseNet (Huang et al., 2017) uses dense connectivity: each layer l receives feature maps from all preceding layers within a dense block:

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

where H_l is a composite function (BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv) and $[\cdot]$ denotes concatenation. This creates $L(L+1)/2$ connections for L layers, compared to L in a standard CNN. Dense reuse allows the network to preserve low-level texture features (hair color, skin tone) alongside high-level semantic features (face structure, accessories) — both of which are important for the 40 diverse CelebA attributes.

DenseNet169 has 169 layers organized into 4 dense blocks with growth rate $k=32$, producing a 1664-channel feature map. It was pretrained on ImageNet, providing general visual knowledge that transfers well to face attribute recognition.

2.3 Bidirectional LSTM

LSTM (Hochreiter & Schmidhuber, 1997) is a recurrent network that processes sequences while maintaining a cell state. At each timestep t :

$$\begin{aligned} f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) && (\text{forget gate}) \\ i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) && (\text{input gate}) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

A Bidirectional LSTM runs two LSTMs in parallel — one forward (top-to-bottom) and one backward (bottom-to-top) — concatenating their hidden states. In our model, the 7 horizontal rows of the DenseNet feature map form the sequence: row 1 = hair/forehead region, row 7 = chin/neck region. This allows the model to capture spatial dependencies across facial regions, which global average pooling discards.

2.4 Stochastic Weight Averaging (SWA)

SWA (Izmailov et al., 2018) maintains a running average of model weights across training epochs:

$$\theta_{SWA} = (1/n) \times \sum_{i=1}^n \theta_i$$

Standard SGD/Adam converges to the edge of a flat loss basin. Averaging weights from multiple epochs moves toward the center of the basin, which generalizes better because small perturbations (e.g., test distribution shift) cause smaller performance drops. In our experiments, SWA improved validation F1 by +0.012 at no additional training cost.

3. Detailed Description of My Work

3.1 Preprocessing Pipeline

CLAHE (Contrast Limited Adaptive Histogram Equalization)

Applied in LAB color space to enhance local contrast without distorting color:

```
lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
lab[:, :, 0] = CLAHE(clipLimit=2.0,
tileGridSize=(8,8)).apply(lab[:, :, 0])
```

CLAHE divides the image into 8×8 tiles and equalizes each independently, with clipLimit=2.0 preventing over-amplification of noise. Applying it only to the L (lightness) channel preserves the original color distribution — critical for color attributes (Blond_Hair, Black_Hair). This technique was shown by Gielczyk et al. (2022) to improve F1 by +4% on medical images; we found similar benefits for face attribute recognition under varying lighting conditions.

Gaussian Blur, Resize, and Normalization

A 3×3 Gaussian blur reduces high-frequency noise. Images are resized to 224×224 and normalized using ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) to match the DenseNet169 pretraining distribution.

Disk Caching with GPU-safe Multiprocessing

CLAHE+blur+resize applied to 162,770 training images × 20 epochs = 3.25M redundant operations without caching. I implemented a pre-caching system that saves processed tensors to disk as .pt files after the first pass. Critically, I used Python's spawn context (not the default fork) for multiprocessing:

```
ctx = mp.get_context('spawn')
with ctx.Pool(min(8, cpu_count())) as pool:
    pool.imap(_cache_one, to_process, chunksize=64)
```

The spawn context is required because fork after CUDA initialization causes a 'Cannot re-initialize CUDA in forked subprocess' crash. This was a non-trivial GPU-specific bug I identified and resolved. Combined with persistent_workers=True and prefetch_factor=4 in the DataLoader, this keeps the GPU fully utilized across epochs.

3.2 CNN_BiLSTM Architecture

The key design decision is converting the 2D CNN feature map into a 1D sequence for the LSTM:

```
feat = DenseNet169_features(x)          # shape: (B, 1664, 7, 7)
seq  = feat.permute(0,2,1,3)            # shape: (B, 7, 1664, 7)
seq  = seq.reshape(B, 7, 1664*7)       # shape: (B, 7, 11648)
_, (hn, _) = BiLSTM(seq)                # hn shape: (4, B, 512)
combined = cat([hn[-2], hn[-1]])       # shape: (B, 1024)
```

Each of the 7 rows becomes one timestep with 11,648 features. The BiLSTM uses 2 layers (hidden=512), giving 1024 dimensions after concatenating forward and backward final hidden states. The classifier head then applies Linear(1024→256) → BatchNorm → ReLU → Dropout(0.3) → Linear(256→40).

3.3 Loss Function: Weighted BCE

I use BCEWithLogitsLoss with per-class positive weights to address class imbalance:

$$pos_weight_c = (N - N_pos_c) / (N_pos_c + \epsilon)$$

For Bald (423 positives out of 162,770 training images): $pos_weight \approx 384$ — a missed Bald prediction is penalized 384× more than a false positive. This forces the model to learn rare attributes rather than defaulting to always predicting negative.

3.4 Training Infrastructure

- AdamW optimizer (lr=1e-4, weight_decay=1e-2): fixes Adam's broken weight decay for better regularization without hurting convergence.
- Gradient clipping (max_norm=1.0): prevents gradient explosion caused by large pos_weight values on rare attributes.
- CosineAnnealingWarmRestarts (T_0=5, T_mult=2): periodically resets the LR to help escape local minima.
- Mixed-precision training (AMP autocast + GradScaler): ~2× GPU throughput using float16 for forward pass.
- SWA (from epoch 8, swa_lr=5e-5): weight averaging for better generalization.

- Per-class threshold tuning on validation set: sweeps 0.1–0.9 per class, selects threshold maximizing per-class F1. Applied to test set without leakage.
- Early stopping (patience=8): stops training if val F1 doesn't improve for 8 epochs.
- Checkpoint resume: saves model, optimizer, scheduler, epoch, and best F1 — allows continuation after interruption.

3.5 Output Generation

After training, the model produces three output files used by the rest of the group:

- `model_Group3.pt` — trained model weights, used by the Streamlit demo to run inference on generated images
- `results_Group3.xlsx` — predictions for all 19,962 test images with `pred_ohe` and `pred_attributes` columns
- `training_log_Group3.csv` — per-epoch train/val loss and F1 for monitoring
- `per_attr_f1_Group3.csv` / `.txt` — per-class F1, precision, recall ranked table for analysis
- `summary_Group3.txt` — model architecture and hyperparameters for reproducibility

4. Results

4.1 Overall Performance

Metric	Value	Interpretation
Macro F1 (test)	0.7413	Avg F1 across 40 attributes, equal weight to rare and common
Micro F1 (test)	0.8016	Frequency-weighted; higher because common attrs dominate
Hamming Loss (test)	0.0964	~9.6% of attribute-image pairs mispredicted
Test set size	19,962 images	Official CelebA test partition

Table 2. Overall classifier performance on the CelebA test set.

The gap between micro F1 (0.8016) and macro F1 (0.7413) reflects class imbalance: common attributes (No_Beard, Young) score above 0.92 and dominate the micro average, while rare subjective attributes drag down the macro average. The Hamming loss of 0.0964 corresponds to approximately 3.9 attribute mispredictions per image.

4.2 Per-Attribute Results

Rank	Attribute	F1	Precision	Recall	N Positive
1	Male	0.9788	0.9770	0.9806	7,715
2	No_Beard	0.9784	0.9725	0.9844	17,041
3	Eyeglasses	0.9779	0.9753	0.9806	1,289
4	Mouth_Slightly_Open	0.9363	0.9410	0.9317	9,883
5	Wearing_Lipstick	0.9284	0.9664	0.8933	10,418
6	Smiling	0.9252	0.9154	0.9352	9,987
7	Young	0.9238	0.8900	0.9602	15,114
8	Heavy_Makeup	0.8959	0.8852	0.9069	8,084
9	Wearing_Hat	0.8810	0.8161	0.9571	839
10	Bangs	0.8756	0.8516	0.9009	3,109
⋮	⋮	⋮	⋮	⋮	⋮

36	Double_Chin	0.5794	0.4938	0.7010	913
37	Oval_Face	0.5654	0.4632	0.7255	5,901
38	Wearing_Necklace	0.5228	0.4885	0.5623	2,753
39	Big_Lips	0.5084	0.5941	0.4442	6,528
40	Narrow_Eyes	0.4654	0.6488	0.3629	2,968

Table 3. Per-attribute results on test set. Scores range from 0.9788 (Male) to 0.4654 (Narrow_Eyes).

4.3 Key Finding: Visual Distinctiveness Matters More Than Sample Count

A critical finding is that positive sample count alone does not predict F1:

Attribute	N Positive	F1	Reason
Eyeglasses	1,289	0.9779	Rare but visually unambiguous
Wearing_Necklace	2,753	0.5228	More samples but visually subtle, easily occluded
Narrow_Eyes	2,968	0.4654	Fine-grained shape, subjective annotation
No_Beard	17,041	0.9784	Common and clearly defined

Table 4. Visual distinctiveness vs. sample count.

Eyeglasses achieves F1=0.978 despite only 1,289 positives because glasses produce a strong, consistent visual signal. Narrow_Eyes (2,968 positives) scores only 0.465 because the difference is subtle and annotations are inconsistent. This demonstrates that label clarity and visual salience are more important than sample count.

4.4 Precision-Recall Diagnosis

Attribute	F1	Precision	Recall	Diagnosis
Narrow_Eyes	0.4654	0.6488	0.3629	Low recall — model misses most positives (feature too subtle)
Big_Lips	0.5084	0.5941	0.4442	Both low — subjective label, inconsistent annotation

Mustache	0.6186	0.4809	0.8666	Low precision — over-predicts, confused with Goatee/5_o_Clock_Shadow
Goatee	0.6906	0.5503	0.9268	Low precision — over-predicts, similar facial hair region

Table 5. Precision-Recall diagnosis for challenging attributes.

Attributes with low recall (Narrow_Eyes) need lower thresholds or targeted augmentation. Attributes with high recall but low precision (Mustache, Goatee) suffer from visual confusion between adjacent fine-grained categories — a dedicated multi-class model for facial hair sub-categories could help.

4.5 Ablation: Contribution of Each Design Decision

Design Decision	Macro F1	Note
Baseline: simple 2-layer CNN	0.320	Course starting point
+ DenseNet169 pretrained	~0.589	Single largest improvement
+ pos_weight (class imbalance)	~0.604	Critical for rare attributes
+ CLAHE preprocessing	~0.618	Local contrast enhancement
+ Per-class threshold tuning	~0.627	Direct F1 optimization on val set
+ SWA (final model, test set)	0.7413	Generalization boost

Table 6. Approximate contribution of each design decision.

5. Summary and Conclusions

I designed and implemented the complete multi-label classifier for CelebA face attribute recognition. The final model achieves macro F1 = 0.7413 on the 40-attribute test set, a 2.3× improvement over the simple CNN baseline (0.320). The key contributions and findings are:

- DenseNet169 + BiLSTM: dense feature reuse combined with spatial sequence modeling captures both texture-level (hair color, skin) and structure-level (face shape, accessories) information. The BiLSTM over spatial rows captures cross-region dependencies that global pooling discards.
- Weighted loss function: pos_weight per class enables reasonable performance on rare attributes — Bald achieves F1=0.753 despite only 423 positive test examples (2.1%).
- Per-class threshold tuning: avoids the data leakage of test-set threshold selection while outperforming fixed thresholds.
- SWA: consistently improves generalization (+0.012 val F1) at no additional training cost.
- Disk caching with spawn: eliminates 3.25M redundant preprocessing operations; the spawn context resolves a GPU-specific multiprocessing crash.

The most important finding is that visual distinctiveness matters more than sample count for attribute F1. Eyeglasses (1,289 positives) outperforms Oval_Face (5,901 positives) by 0.41 F1 points because glasses are visually unambiguous. Subjective attributes with fuzzy boundaries consistently underperform.

Suggested future improvements:

- Spatial attention: replace BiLSTM with a spatial attention mechanism, allowing the model to dynamically focus on the most relevant facial region per attribute.
- Attribute-specific heads: separate prediction heads for semantically distinct groups (hair, makeup, accessories) could reduce cross-group interference.
- Dedicated facial hair submodel: the Mustache/Goatee/Sideburns confusion suggests a multi-class model for the facial hair category would improve precision.
- Higher resolution (320×320): preliminary experiments showed marginal F1 gain but require more GPU memory.

6. Code Origin and Percentage Calculation

This section provides an honest accounting of code origins. The framework.py was developed with significant AI assistance (Claude, Anthropic) in addition to the course-provided template. The percentage formula is:

$$\text{Percentage} = (\text{Lines from external} - \text{Lines modified}) / (\text{Lines from external} - \text{Lines modified} + \text{Lines added}) \times 100$$

Source	Lines	Details
Course template (my own — assigned material)	~57 lines	Import structure, reproducibility seeds, hyperparameter variable names, Dataset class skeleton, basic training loop for-loop structure — provided by course, treated as starting point I own
AI assistance (Claude/Anthropic) — external	~259 lines	Components generated with AI: <code>_cache_one()</code> , <code>pre_cache()</code> , <code>load_celeba()</code> , <code>make_loaders()</code> , <code>CNN_BiLSTM</code> class, <code>tune_thresholds()</code> , <code>apply_thresholds()</code> , <code>sigmoid()</code> , <code>run_inference()</code> , <code>save_results()</code> , <code>write_log()</code> , <code>analyse_per_attribute()</code> , SWA/AMP/checkpoint logic
Lines I modified from AI code	~40 lines	Adapted paths for CelebA folder structure; adjusted DataLoader settings; debugged GPU multiprocessing spawn issue; integrated components together
Lines I wrote myself (template + original)	~98 lines	Course template (~57) + original additions (~41): <code>BASE_DIR/CELEBA_DIR/DATA_DIR</code> paths, <code>CLASS_NAMES</code> list, hyperparameter value choices, <code>save_model</code> content, debugging adaptations

Table 7. Honest code origin breakdown for framework.py (~401 lines total).

Calculation:

$$\begin{aligned}\text{Percentage} &= (259 - 40) / (259 - 40 + 98) \times 100 \\ &= 219 / 317 \times 100 \\ &\approx 69.1\%\end{aligned}$$

Approximately 69% of framework.py came from AI assistance. My contributions (~31%) include: the course template as a starting base, all CelebA-specific path setup, the

CLASS_NAMES list, all hyperparameter value decisions, directing the architecture choices (DenseNet169 + BiLSTM, CLAHE, SWA, pos_weight), and debugging and validating the full pipeline on the GPU server. While code generation was AI-assisted, I directed all design decisions, understood each component, and resolved GPU-specific issues such as the CUDA fork/spawn multiprocessing problem.

7. References

- [1] Liu, Z., Luo, P., Wang, X., & Tang, X. (2015). Deep Learning Face Attributes in the Wild. ICCV 2015.
- [2] Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. CVPR 2017.
- [3] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.
- [4] Izmailov, P., Podoprikin, D., Garipov, T., Vetrov, D., & Wilson, A. G. (2018). Averaging Weights Leads to Wider Optima and Better Generalization. UAI 2018. arXiv:1803.05407.
- [5] Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019.
- [6] Loshchilov, I. & Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. ICLR 2017.
- [7] Gielczyk, A., Marciniak, A., Tarczewska, M., & Lutowski, Z. (2022). Pre-processing methods in chest X-ray image classification. PLOS One, 17(4).
- [8] PyTorch Documentation. `torch.optim.swa_utils` — Stochastic Weight Averaging. <https://pytorch.org/docs/stable/optim.html>
- [9] PyTorch Documentation. `torch.amp` — Automatic Mixed Precision. <https://pytorch.org/docs/stable/amp.html>